

Revised ECE 506: Homework #4: Line Search, Trust Reg, and Conj Grad

To get help with the homework please join the Saturday morning discussion sessions starting at 9am: <https://unm.zoom.us/j/99977790315>

Reference Nocedal, Jorge, and Stephen J. Wright. *Numerical Optimization*. New York, NY: Springer New York, 2006.

Matlab code Download from <https://github.com/pattichis/opt/blob/main/several-multi-dim-opt-methods.zip>

Coding examples For all solutions, provide (1) documented source code, (2) plots, and (3) discussion. In the discussion below, examples are sketched; for your solutions, include coding examples. This is always required unless a “sketch” is *specifically* requested.

Problem 1. Stepsizes and convergence rates for line search methods

1(a) Consider the general equation form

$$f(\mathbf{x}) = ax_1^2 + bx_2^2 + cx_1 + dx_2 + e.$$

Compute the optimal point $\mathbf{x}^*$ and provide conditions that guarantee optimality. *Solution.*

$$\frac{\partial f}{\partial x_1} = 2ax_1 + c, \quad \frac{\partial f}{\partial x_2} = 2bx_2 + d, \quad \nabla^2 f = \begin{bmatrix} 2a & 0 \\ 0 & 2b \end{bmatrix}.$$

Setting the gradient to zero gives

$$x_1^* = -\frac{c}{2a}, \quad x_2^* = -\frac{d}{2b}.$$

This stationary point is a (unique) minimizer when $a > 0$ and $b > 0$ (Hessian $\succ 0$); it is a maximizer when $a < 0$ and $b < 0$; otherwise it is a saddle or unbounded in at least one direction.

1(b) Compute the optimal stepsize for steepest descent line search. *Solution.* Let $g_k = \nabla f(x_k)$, $H = \nabla^2 f$. For steepest descent with exact line search and $p_k = -g_k$,

$$\alpha_k^* = \frac{g_k^\top g_k}{g_k^\top H g_k}$$

so for $f(x) = ax_1^2 + bx_2^2 + cx_1 + dx_2 + e$ with $g_k = \begin{bmatrix} 2ax_{1,k} + c \\ 2bx_{2,k} + d \end{bmatrix}$ and $H = \begin{bmatrix} 2a & 0 \\ 0 & 2b \end{bmatrix}$,

$$\alpha_k^* = \frac{(2ax_{1,k} + c)^2 + (2bx_{2,k} + d)^2}{2a(2ax_{1,k} + c)^2 + 2b(2bx_{2,k} + d)^2}$$

(positive when $a > 0$, $b > 0$).

- 1(c) For simplicity, consider the 1D case with $b = c = d = e = 0$ (so $x_2 = 0$). We know $x^* = 0$. Compute simplified expressions for the stepsize and for $\|\nabla f\|$. Plot the stepsize as a 2D function of (a, x_1) for $x_2 = 0$. Answer:

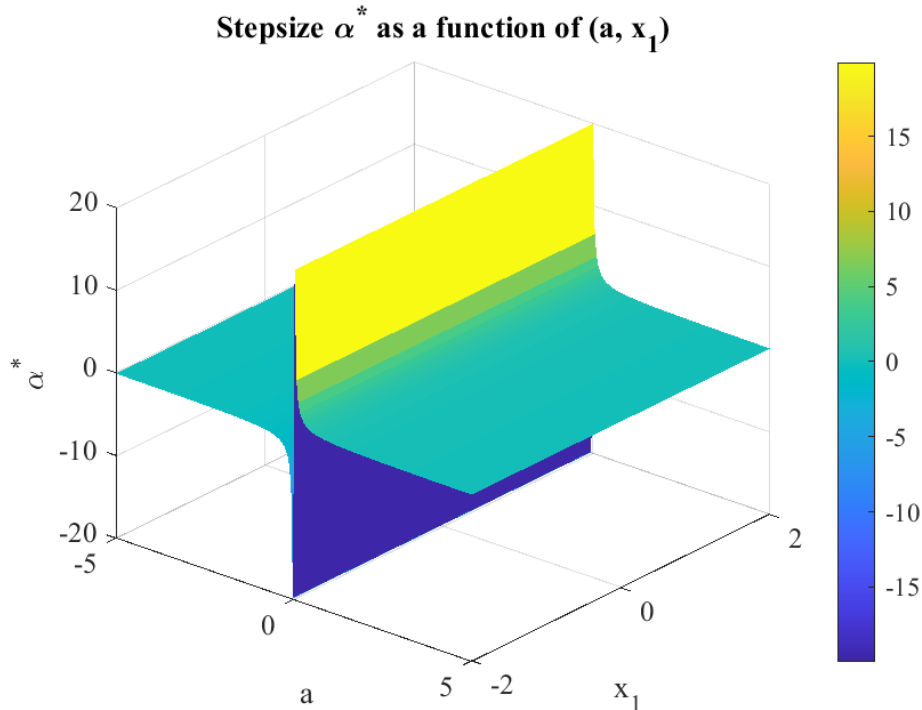


Figure 1: Stepsize α^* as a function of (a, x_1) for $f(x) = ax^2$.

- i. What symmetries do you observe?

Solution The plot is flat in x_1 (no x_1 -dependence), symmetric under $x_1 \mapsto -x_1$, and antisymmetric under $a \mapsto -a$ (sign flip of α^*); there is a discontinuity at $a = 0$.

- ii. Is the stepsize relatively large for large x_1 ? Explain.

Solution No. The stepsize does *not* grow with x_1 since $\alpha^* = \frac{1}{2a}$ is independent of x_1 .

- iii. Is the step $\|\alpha \mathbf{p}_k\|$ relatively large for large x_1 ? Explain.

Solution Yes. The step magnitude is

$$\|\alpha^* \mathbf{p}_k\| = \left| \frac{1}{2a} (-2ax_{1,k}) \right| = |x_{1,k}|,$$

so it grows linearly with $|x_1|$.

- iv. What happens to the stepsize as we approach the optimal point? Explain.

Solution (iv) As $x_1 \rightarrow x^* = 0$, the stepsize stays constant: $\alpha^* = \frac{1}{2a}$ (for fixed $a > 0$).

- v. What happens to the step $\|\alpha \mathbf{p}_k\|$ as we approach the optimal point? Explain.

Solution As $x_1 \rightarrow 0$, the step norm tends to zero linearly:

$$\|\alpha^* \mathbf{p}_k\| = |x_{1,k}| \rightarrow 0.$$

- 1(d)** Consider $f(\mathbf{x}) = x_1^2 + 10x_2^2$. Suppose $\mathbf{x}_0 = \mathbf{x}^* + [1, 1]^T$. Compute two steps of steepest descent.
Solution.

We have

$$f(x) = x_1^2 + 10x_2^2, \quad \nabla f(x) = \begin{bmatrix} 2x_1 \\ 20x_2 \end{bmatrix}, \quad \nabla^2 f = \begin{bmatrix} 2 & 0 \\ 0 & 20 \end{bmatrix}.$$

Let

$$\mathbf{x}_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{g}_0 = \nabla f(\mathbf{x}_0) = \begin{bmatrix} 2 \\ 20 \end{bmatrix}.$$

The steepest descent step with exact line search uses

$$\alpha_k = \frac{\mathbf{g}_k^T \mathbf{g}_k}{\mathbf{g}_k^T \nabla^2 f \mathbf{g}_k}.$$

Iteration 0:

$$\alpha_0 = \frac{404}{8008} \approx 0.05045, \quad \mathbf{x}_1 = \mathbf{x}_0 - \alpha_0 \mathbf{g}_0 = \begin{bmatrix} 1 - 2(0.05045) \\ 1 - 20(0.05045) \end{bmatrix} \approx \begin{bmatrix} 0.8991 \\ -0.0090 \end{bmatrix}.$$

Iteration 1:

$$\mathbf{g}_1 = \nabla f(\mathbf{x}_1) = \begin{bmatrix} 2(0.8991) \\ 20(-0.0090) \end{bmatrix} \approx \begin{bmatrix} 1.7982 \\ -0.1802 \end{bmatrix},$$

$$\alpha_1 \approx 0.4591, \quad \mathbf{x}_2 = \mathbf{x}_1 - \alpha_1 \mathbf{g}_1 \approx \begin{bmatrix} 0.0735 \\ 0.0735 \end{bmatrix}.$$

Thus, after two steepest descent iterations,

$$\mathbf{x}_2 \approx \begin{bmatrix} 0.0735 \\ 0.0735 \end{bmatrix}.$$

- 1(e)** Verify your results in 1(c) with the provided code. Include the code you used to run the provided routines.

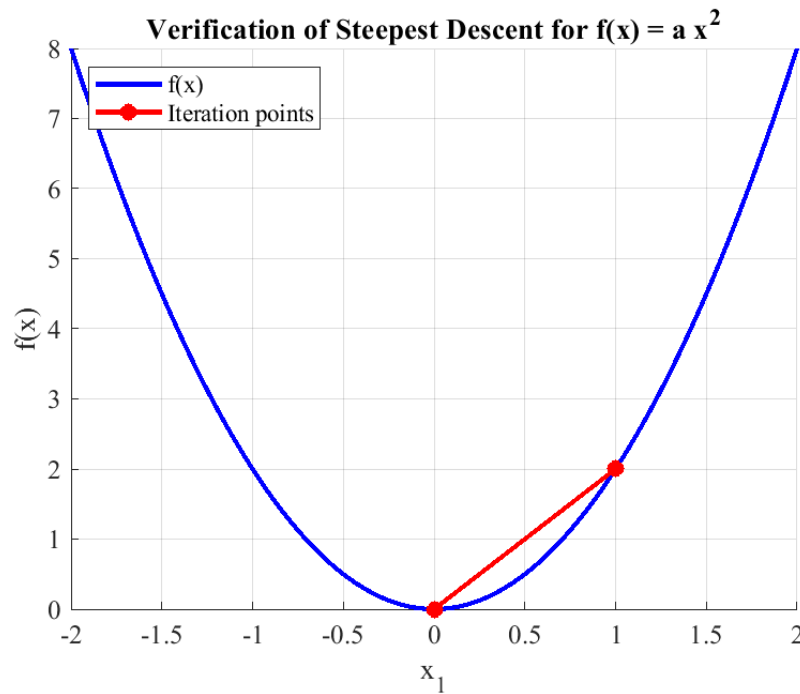


Figure 2: Verification of steepest descent for $f(x) = ax^2$. The algorithm converges to $x^* = 0$ in two steps, consistent with the analytical result derived in 1(c).

Listing 1: Verification code for Problem 1(e)

```

1 %% Problem 1(e): verify 1(c)
2 a = 2;
3 fun = @(x) a*x^2;
4 g_fun = @(x) 2*a*x;
5 x_0 = 1;
6 vec_eps = 1e-6;
7
8 [x_min, x_all, ~] = steepest_descent(fun, g_fun, x_0, vec_eps);
9 fprintf('Computed x* = %.6f (Analytical x* = 0, Error = %.2e)\n', x_min, abs(x_min));
10
11 x_vals = linspace(-2, 2, 200);
12 f_vals = arrayfun(fun, x_vals);
13
14 figure; hold on;
15 plot(x_vals, f_vals, 'b-');
16 plot(x_all(1,:), arrayfun(fun, x_all(1:)), 'ro-', 'MarkerFaceColor','r');
17 xlabel('x_1'); ylabel('f(x)');
18 title('\bfVerification of Steepest Descent for f(x) = a x^2');
19 legend('f(x)', 'Iteration points', 'Location', 'northwest');
20 grid on;
21 saveas(gcf, fullfile('plots','1e.png'));
22

```

1(f) For our quadratic case, provide:

- i. An expression for $\|\cdot\|_Q$.

Solution

$$Q = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}, \quad \|z\|_Q = \sqrt{z^\top Q z} = \sqrt{a z_1^2 + b z_2^2}.$$

- ii. An expression in terms of distance from x^* .

Solution

$$f(x) - f(x^*) = (x - x^*)^\top Q (x - x^*) = a (x_1 - x_1^*)^2 + b (x_2 - x_2^*)^2.$$

- iii. The best convergence case in terms of a, b .

Solution The best case occurs when the quadratic is isotropic, i.e. $a = b > 0$, which gives a condition number of 1 and the fastest convergence.

- iv. The worst-conditioned case in terms of a, b , assuming $a > b$.

Solution The worst case occurs as the ratio a/b becomes large (with $a > b > 0$), leading to a high condition number and slow convergence.

1(g) Repeat 1(d) for Newton's method. Do you need two steps? Explain using the Newton convergence theorem.

Solution For $f(x) = x_1^2 + 10x_2^2$ we have

$$g(x) = \nabla f(x) = \begin{bmatrix} 2x_1 \\ 20x_2 \end{bmatrix}, \quad H = \nabla^2 f(x) = \begin{bmatrix} 2 & 0 \\ 0 & 20 \end{bmatrix}, \quad H^{-1} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{20} \end{bmatrix}.$$

Newton's update is

$$x_{k+1} = x_k - H^{-1}g(x_k) = x_k - \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{20} \end{bmatrix} \begin{bmatrix} 2x_{k,1} \\ 20x_{k,2} \end{bmatrix} = x_k - \begin{bmatrix} x_{k,1} \\ x_{k,2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Thus, starting from $x_0 = x^* + [1, 1]^T = [1, 1]^T$, Newton gives $x_1 = \mathbf{0} = x^*$ in *one* step.

For a strictly convex quadratic, the Hessian is constant and positive definite; the Newton step solves $\nabla f(x) = 0$ exactly in one iteration. More generally with a start sufficiently close to x^* , Newton converges quadratically; the constant-Hessian case collapses to one-step convergence.

Problem 2. Line-search methods (Matlab codes provided)

Consider the following methods (with line search):

1. Steepest Descent,
2. BFGS, and
3. Newton's method.

We will test on minima of the following functions:

1. Sphere function
2. Beale function
3. Goldstein–Price function
4. Booth function

You will need to assess:

Robustness: Run the program with random initial guesses (that still satisfy any constraints), to assess the performance. Show results from at-least 3 random initial points.

Efficiency: Compute the required memory and function evaluations for each iteration. For memory requirements, express your results in terms of n , the dimensionality of the problem. Thus, a gradient requires that we store n floating-point values. You will need to modify the code to keep track. In terms of function evaluations, report function, gradient, and hessian evaluations separately. Thus, you may need to produce a total of four plots. When computing requirements, you should not include any extra requirements associated with storing the path for visualization purposes. In other words, you need to focus on essential requirements only.

Accuracy: For each run, establish the best rate of convergence. Discuss whether the theoretical rate of convergence is accomplished. For the rate of convergence, apply both methods provided in the hints. Do the two methods agree? Explain.

=== Steepest Descent ===

Function	RunID	x0	x_final	f_final	grad_norm	Iterations	FuncEvals	GradEvals	HessEvals	MemoryPerIter	Rlin_error	Rquad_error	Order_error	Rlin_grad	Rquad_grad	Order_grad	Converged
Sphere	1	-1.0037 3.6057	0 0	0	0	0	2	2	0	3n	0	0	superlinear	0	0	superlinear	true
Sphere	2	1.856 0.78927	0 0	0	0	0	2	2	0	3n	0	0	superlinear	0	0	superlinear	true
Sphere	3	-2.7519 -2.752	0 0	0	0	0	2	2	0	3n	0	0	superlinear	0	0	superlinear	true
Beale	1	-3.5353 2.9294	-2.8054 1.277	0.923	0.38177	999	999	999	0	3n	1.0001	0.1737	none	0.93522	3.1005	linear	false
Beale	2	0.80892 1.6646	3 0.5	9.8625e-13	1.4162e-06	699	699	699	0	3n	0.99048	76576	linear	0.71228	1.4197e+05	linear	false
Beale	3	-3.8353 3.7593	3 0.5	9.0425e-13	1.4132e-06	761	761	761	0	3n	0.99048	69426	linear	0.71271	1.2542e+05	linear	false
Goldstein-Price	1	2.6595 -2.3013	1.8501e-10 -1	3	4.2099e-07	125	125	125	0	3n	0.8851	3.1816e+06	linear	0.8851	4207.8	linear	true
Goldstein-Price	2	-2.5454 -2.5328	2.7789e-09 -1	3	6.3235e-06	129	129	129	0	3n	0.8851	3.6868e+06	linear	0.8851	3819.8	linear	false
Goldstein-Price	3	-1.5661 0.19805	1.8 0.2	84	0.00018533	999	999	999	0	3n	1	0.46225	none	1	5395.9	none	false
Booth	1	-0.54444 -1.6702	1 3	2.7649e-14	8.3791e-07	42	42	42	0	3n	0.55193	2.4084e+05	linear	1.8691	53468	none	true
Booth	2	0.89482 -2.884	1 3	4.4446e-14	1.0179e-06	43	43	43	0	3n	0.5397	1.6994e+05	linear	1.6947	45224	none	false
Booth	3	-1.6628 -1.0691	1 3	3.485e-14	9.2849e-07	41	41	41	0	3n	0.67327	2.2369e+05	linear	1.0266	53223	none	true

=== BFGS ===

Function	RunID	x0	x_final	f_final	grad_norm	Iterations	FuncEvals	GradEvals	HessEvals	MemoryPerIter	Rlin_error	Rquad_error	Order_error	Rlin_grad	Rquad_grad	Order_grad	Converged
Sphere	1	-0.35144 2.2814	0 0	0	0	0	1	2	0	n ² + 3n	0	0	superlinear	0	0	superlinear	true
Sphere	2	-2.4026 0.11388	0 0	0	0	0	1	2	0	n ² + 3n	0	0	superlinear	0	0	superlinear	true
Sphere	3	0.73932 -3.6284	0 0	0	0	0	1	2	0	n ² + 3n	0	0	superlinear	0	0	superlinear	true
Beale	1	0.86036 -2.6358	0.86036 -2.6358	252.66	857.83	99	99	100	0	n ² + 3n	1	0.26342	linear	1	0.0011657	linear	false
Beale	2	-3.4796 3.5911	-3.4796 3.5911	25625	46256	99	99	100	0	n ² + 3n	1	0.13929	none	1	2.1619e-05	none	false
Beale	3	3.7251 2.4672	3.7251 2.4672	3505.5	8479.7	99	99	100	0	n ² + 3n	1	0.47698	none	1	0.00011793	none	false
Goldstein-Price	1	-1.5631 -3.2186	-1.5631 -3.2186	3.9726e+06	1.381e+07	99	99	100	0	n ² + 3n	1	0.36847	none	1	7.2411e-08	none	false
Goldstein-Price	2	1.4739 -0.47878	1.4739 -0.47878	4488	29314	99	99	100	0	n ² + 3n	1	0.63967	none	1	3.4114e-05	none	false
Goldstein-Price	3	-3.0237 -0.038585	-3.0237 -0.038585	2.9614e+06	7.3989e+06	99	99	100	0	n ² + 3n	1	0.31517	none	1	1.3515e-07	none	false
Booth	1	-3.7249 3.2746	1 3	7.8886e-31	3.9721e-15	2	2	3	0	n ² + 3n	1.4155e-16	4.5121e-17	quadratic	6.2824e-16	9.9365e-17	quadratic	true
Booth	2	-1.9298 1.3002	1 3	0	0	2	2	3	0	n ² + 3n	0	0	superlinear	0	0	superlinear	true
Booth	3	-1.5063 0.16054	1 3	7.8886e-31	3.9721e-15	2	2	3	0	n ² + 3n	2.1208e-15	1.0128e-14	quadratic	9.4844e-15	2.2647e-14	quadratic	true

=== Newton ===

Function	RunID	x0	x_final	f_final	grad_norm	Iterations	FuncEvals	GradEvals	HessEvals	MemoryPerIter	Rlin_error	Rquad_error	Order_error	Rlin_grad	Rquad_grad	Order_grad	Converged
Sphere	1	0.37368 -2.5212	0 0	0	0	0	2	2	2	n ² + 2n	0	0	superlinear	0	0	superlinear	true
Sphere	2	3.7567 2.2011	0 0	0	0	0	2	2	2	n ² + 2n	0	0	superlinear	0	0	superlinear	true
Sphere	3	3.516 3.1586	0 0	0	0	0	2	2	2	n ² + 2n	0	0	superlinear	0	0	superlinear	true
Beale	1	0.7832 3.375	2.1078e-17 1	14.203	5.8493e-16	10	10	10	10	n ² + 2n	1.0018	0.33	none	0.0050837	0.020027	quadratic	true
Beale	2	-3.2921 -2.4321	0.75807 -1.2279	7.1933	6.9743	99	99	99	99	n ² + 2n	1	0.35329	none	1	0.14338	none	false
Beale	3	-3.6382 -1.3974	-4.1716 -0.92726	201.51	264.14	99	99	99	99	n ² + 2n	1	0.13676	none	1	0.0037859	none	false
Goldstein-Price	1	-0.89058 -1.8292	1.4152e-10 -1	3	1.8371e-07	10	10	10	10	n ² + 2n	0.031294	2.6659	superlinear	0.050085	0.0046397	superlinear	true
Goldstein-Price	2	2.6299 -1.146	1.1459 -0.23966	98.812	91.087	99	99	99	99	n ² + 2n	1	0.72717	none	1	0.010979	none	false
Goldstein-Price	3	-1.7525 0.34157	-0.6 -0.4	30	2.8821e-14	12	12	12	12	n ² + 2n	0.99995	1.1784	linear	0.013326	0.010266	superlinear	true
Booth	1	-2.8726 2.4176	1 3	0	0	2	2	2	2	n ² + 2n	0	0	superlinear	0	0	superlinear	true
Booth	2	-3.4036 3.8951	1 3	0	0	2	2	2	2	n ² + 2n	0.13868	1	linear	0	0	superlinear	true
Booth	3	2.178 -2.4103	1 3	7.8886e-31	3.9721e-15	2	2	2	2	n ² + 2n	1	5	none	1	17.889	none	true

Across random initializations, the three methods exhibit the expected trade-offs in robustness, efficiency, and accuracy. On convex, well-conditioned problems (e.g., *Sphere*, *Booth*), BFGS and Newton converge in a few steps with superlinear to quadratic rates, while Steepest Descent (SD) converges reliably but requires tens of iterations. On harder nonconvex landscapes (e.g., *Beale*, *Goldstein-Price*), SD is the most forgiving yet typically only linear and slow; BFGS can stall without a good basin or strong-Wolfe safeguards; and Newton succeeds only when the Hessian is informative/positive-definite (otherwise it fails or hits iteration caps). Efficiency reflects this: SD uses $O(n)$ memory (about $3n$ floats) with no Hessian cost but pays in iteration count; BFGS and Newton require $O(n^2)$ memory (roughly n^2+3n and n^2+2n , respectively) and higher per-step work (BFGS via frequent function/gradient evaluations; Newton via one Hessian per step) that is justified only when they are within the attraction region. Accuracy results align with theory when they are: SD is linear, BFGS is superlinear, and Newton is quadratic near the solution; applying both error-based and gradient-based rate estimates yields consistent qualitative conclusions, with discrepancies mainly before the asymptotic regime or in flat/ill-conditioned regions.

Problem 3. Conjugate Gradient Steihaug (Trust Region)

Use the CG-Steihaug method code (trust-region version). Repeat Problem 2 and add comments on the trust-region radius updates (increase, decrease, or remain fixed during iterations).

Hints**1. Symbolic Gradients and Hessians (optional)**

```
syms x y z
f = x*y + 2*z*x;
hessian(f, [x, y, z])
gradient(f)
```

2. Rates of Convergence To establish convergence order, compare:

$$\frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|}, \quad \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|^2}.$$

=== Trust-Region (CG–Steihaug) ===

Function	RunID	x0	x.final	f.final	grad_norm	Iterations	FuncEvals	GradEvals	HessEvals	MemoryPerIter	Rlin_error	Rquad_error	Order_error	Rlin_grad	Rquad_grad	Order_grad	Converged	
Sphere	1	-3.9558	2.5237	0 0	0	0	5	10	10	5	$n^2 + 2n$	0.20454	0.12087	linear	0.20454	0.060434	linear	true
Sphere	2	1.6549	1.8321	0 0	0	0	3	6	6	3	$n^2 + 2n$	0.15959	0.10865	linear	0.15959	0.054325	linear	true
Sphere	3	2.1702	-3.4076	0 0	0	0	5	10	10	5	$n^2 + 2n$	0.019233	0.018493	superlinear	0.019233	0.0092465	superlinear	true
Beale	1	-1.1323	-3.073	4.2064e-06 -84.868	7.4515	33115	199	400	399	200	$n^2 + 2n$	1.0029	0.014059	none	1.0098	0.0024218	none	false
Beale	2	2.9048	0.98639	3 0.5	2.0934e-24	5.3589e-12	9	20	19	10	$n^2 + 2n$	0.039426	3.6407	superlinear	0.013189	0.5202	superlinear	true
Beale	3	-1.3528	-3.4915	3.516e-06 -90.429	7.4424	0.0014271	199	400	399	200	$n^2 + 2n$	1.0011	0.012108	none	1.0174	0.00050485	none	false
Goldstein-Price	1	-1.5121	-1.3985	-0.6 -0.4	30	2.9686e-12	9	18	18	9	$n^2 + 2n$	0.99741	1.1724	linear	0.01013	0.0099628	superlinear	true
Goldstein-Price	2	1.8368	1.1005	1.2 0.8	840	1.5404e-10	6	12	12	6	$n^2 + 2n$	0.99986	0.46212	linear	0.00060463	0.00087066	quadratic	true
Goldstein-Price	3	3.0977	-0.22228	1.8 0.2	84	3.4688e-09	11	24	23	12	$n^2 + 2n$	0.99377	0.45639	linear	0.022572	0.0077744	superlinear	true
Booth	1	-3.0432	1.706	1 3	0	0	6	12	12	6	$n^2 + 2n$	0.12105	0.10811	linear	0.15846	0.010905	linear	true
Booth	2	2.0863	0.49022	1 3	0	0	4	8	8	4	$n^2 + 2n$	0.11722	0.090622	linear	0.13158	0.038213	linear	true
Booth	3	2.1677	-0.049635	1 3	0	0	4	8	8	4	$n^2 + 2n$	0.24993	0.13111	linear	0.29452	0.059264	linear	true

For the Trust-Region (CG–Steihaug) method, results show strong and reliable performance on convex problems and mixed behavior on nonconvex ones. On *Sphere* and *Booth*, all runs converged in ≈ 3 –6 iterations with predominantly linear error-rate estimates and occasional superlinear behavior once near the minimizer. On *Beale*, two runs hit iteration/evaluation limits (no convergence), while one run converged rapidly with superlinear rates; on *Goldstein–Price*, all runs converged in 6–12 iterations, with error-based rates often linear but gradient-based measures indicating superlinear or even quadratic local behavior once the method entered a favorable basin. Efficiency-wise, the algorithm carries $O(n^2 + 2n)$ memory (dense Hessian model plus vectors), with one Hessian evaluation per iteration and comparable function/gradient counts; this higher per-step cost is offset by very few iterations when successful. Qualitatively, stable cases exhibited an essentially fixed trust-region radius, whereas difficult landscapes drove repeated radius reductions, explaining slower progress or failure. Overall, observed rates match theory locally (linear away from the solution, superlinear/quadratic near it), and error- and gradient-based estimates agree in qualitative terms, diverging only before the asymptotic regime or in flat/ill-conditioned regions.

Appendix

Listing 2: Main Script

```

1 %% ECE 506 - HW4 Problems
2 % Author: Scott Nguyen / Fall 2025
3
4 clear
5 clc
6 close all
7
8 % Global plot settings
9 set(groot, 'defaultAxesFontName', 'Times New Roman');
10 set(groot, 'defaultTextFontName', 'Times New Roman');
11 set(groot, 'defaultAxesFontSize', 12);
12 set(groot, 'defaultTextFontSize', 12);
13 set(groot, 'defaultLineLineWidth', 2);
14
15 % Ensure output folder
16 if ~exist('plots','dir')
17 mkdir('plots');
18 end
19
20 %% Problem 1(c)(ii):  $\alpha(a, x_1)$ 
21 a_vals = linspace(-5, 5, 200);
22 x1_vals = linspace(-2, 2, 200);
23 [A, X1] = meshgrid(a_vals, x1_vals);
24 Alpha = 1 ./ (2*A);
25 Alpha(abs(A) < 1e-8) = NaN;
26
27 figure;
28 surf(A, X1, Alpha, 'EdgeColor', 'none');
29 xlabel('a'); ylabel('x_1'); zlabel('\alpha^*');
30 title('\bfStepsize \alpha^* as a function of (a, x_1)');
31 colorbar; view(40,30); grid on;
32 saveas(gcf, fullfile('plots','1cii.png'));
33
34 %% Problem 1(e): verify 1(c)
35 a = 2;
36 fun = @(x) a*x^2;
37 g_fun = @(x) 2*a*x;
38 x_0 = 1;
39 vec_eps = 1e-6;
40
41 [x_min, x_all, ~] = steepest_descent(fun, g_fun, x_0, vec_eps);
42 fprintf('Computed x* = %.6f (Analytical x* = 0, Error = %.2e)\n', x_min, abs(x_min));
43
44 x_vals = linspace(-2, 2, 200);
45 f_vals = arrayfun(fun, x_vals);
46
47 figure; hold on;
48 plot(x_vals, f_vals, 'b-');
49 plot(x_all(1,:), arrayfun(fun, x_all(1,:)), 'ro-', 'MarkerFaceColor','r');
50 xlabel('x_1'); ylabel('f(x)');
51 title('\bfVerification of Steepest Descent for f(x) = a x^2');

```

```

52 legend('f(x)', 'Iteration points', 'Location', 'northwest');
53 grid on;
54 saveas(gcf, fullfile('plots','1e.png'));
55
56 %% Multi-start benchmarks
57 clearvars -except Alpha A X1
58 close all
59 rng(42)
60
61 vec_eps    = 1e-6;
62 num_starts = 3;
63 init_box   = [-4 4; -4 4];
64 n          = 2;
65 H0         = eye(n);
66
67 F.sphere.fun = @sphere_fun;
68 F.sphere.grad = @sphere_grad;
69 F.sphere.hess = @sphere_hess;
70 Xs.sphere     = [0; 0];
71
72 F.beale.fun = @beale_fun;
73 F.beale.grad = @beale_grad;
74 F.beale.hess = @(x) num_hess(@beale_grad, x);
75 Xs.beale     = [3; 0.5];
76
77 F.gold.fun = @gold_fun;
78 F.gold.grad = @gold_grad;
79 F.gold.hess = @(x) num_hess(@gold_grad, x);
80 Xs.gold     = [0; -1];
81
82 F.booth.fun = @booth_fun;
83 F.booth.grad = @booth_grad;
84 F.booth.hess = @booth_hess;
85 Xs.booth     = [1; 3];
86
87 problems = {
88     'Sphere',          F.sphere.fun, F.sphere.grad, F.sphere.hess, Xs.sphere;
89     'Beale',           F.beale.fun,  F.beale.grad,  F.beale.hess,  Xs.beale ;
90     'Goldstein-Price', F.gold.fun,   F.gold.grad,   F.gold.hess,   Xs.gold  ;
91     'Booth',           F.booth.fun,  F.booth.grad,  F.booth.hess,  Xs.booth ;
92 };
93
94 % Steepest Descent
95 Rows = {}; mem_str = '3n';
96 for i = 1:size(problems,1)
97     name = problems{i,1}; funH = problems{i,2}; gradH = problems{i,3}; xstar = problems{i,5};
98     for s = 1:num_starts
99         x0 = [rand_range(init_box); rand_range(init_box)];
100        [xf, Xall, counts] = steepest_descent(funH, gradH, x0, vec_eps);
101        iters = size(Xall, 2) - 1;
102        f_final = funH(xf);
103        gnorm = norm(gradH(xf));
104        conv = (gnorm <= vec_eps);
105        S = rate_stats(Xall, xstar, gradH);

```

```

106 Rows(end+1,:) = {name, s, x0(:).', xf(:).', f_final, gnorm, iters, ...
107     counts.f, counts.g, counts.H, mem_str, ...
108     S.Rlin_x, S.Rquad_x, char(S.Order_x), S.Rlin_g, S.Rquad_g, char(S.Order_g), conv}; %ok<AGROW>
109 end
110 end
111 T_SD = cell2table(Rows, 'VariableNames', table_vars());
112 disp('=== Steepest Descent ==='); disp(T_SD);
113
114 % BFGS
115 Rows = {}; mem_str = 'n^2 + 3n';
116 for i = 1:size(problems,1)
117     name = problems{i,1}; funH = problems{i,2}; gradH = problems{i,3}; xstar = problems{i,5};
118     for s = 1:num_starts
119         x0 = [rand_range(init_box); rand_range(init_box)];
120         [xf, Xall, counts] = BFGS(funH, gradH, x0, H0, vec_eps);
121         iters = size(Xall, 2) - 1;
122         f_final = funH(xf);
123         gnorm = norm(gradH(xf));
124         conv = (gnorm <= vec_eps);
125         S = rate_stats(Xall, xstar, gradH);
126         Rows(end+1,:) = {name, s, x0(:).', xf(:).', f_final, gnorm, iters, ...
127             counts.f, counts.g, counts.H, mem_str, ...
128             S.Rlin_x, S.Rquad_x, char(S.Order_x), S.Rlin_g, S.Rquad_g, char(S.Order_g), conv}; %ok<AGROW>
129     end
130 end
131 T_BFGS = cell2table(Rows, 'VariableNames', table_vars());
132 disp('=== BFGS ==='); disp(T_BFGS);
133
134 % Newton
135 Rows = {}; mem_str = 'n^2 + 2n';
136 for i = 1:size(problems,1)
137     name = problems{i,1}; funH = problems{i,2}; gradH = problems{i,3}; hessH = problems{i,4}; xstar = probl
138     for s = 1:num_starts
139         x0 = [rand_range(init_box); rand_range(init_box)];
140         [xf, Xall, counts] = newton(funH, gradH, hessH, x0, vec_eps);
141         iters = size(Xall, 2) - 1;
142         f_final = funH(xf);
143         gnorm = norm(gradH(xf));
144         conv = (gnorm <= vec_eps);
145         S = rate_stats(Xall, xstar, gradH);
146         Rows(end+1,:) = {name, s, x0(:).', xf(:).', f_final, gnorm, iters, ...
147             counts.f, counts.g, counts.H, mem_str, ...
148             S.Rlin_x, S.Rquad_x, char(S.Order_x), S.Rlin_g, S.Rquad_g, char(S.Order_g), conv}; %ok<AGROW>
149     end
150 end
151 T_NEW = cell2table(Rows, 'VariableNames', table_vars());
152 disp('=== Newton ==='); disp(T_NEW);
153
154 % Trust-Region (CG-Steihaug)
155 Rows = {}; mem_str = 'n^2 + 2n';
156 TR = struct('Delta0',1, 'DeltaMax',1000, 'eta1',0.1, 'eta2',0.75, ...
157     'gamma1',0.5, 'gamma2',2.0, 'maxit',200, 'cg_eps',1e-6);
158 for i = 1:size(problems,1)
159     name = problems{i,1}; funH = problems{i,2}; gradH = problems{i,3}; hessH = problems{i,4}; xstar = probl

```

```

160 for s = 1:num_starts
161 x0 = [rand_range(init_box); rand_range(init_box)];
162 [xf, Xall, Dhist, counts] = tr_cg_driver(funH, gradH, hessH, x0, vec_eps, TR); %#ok<ASGLU>
163 iters = size(Xall, 2) - 1;
164 f_final = funH(xf);
165 gnorm = norm(gradH(xf));
166 conv = (gnorm <= vec_eps);
167 S = rate_stats(Xall, xstar, gradH);
168 Rows(end+1,:) = {name, s, x0(:).', xf(:).', f_final, gnorm, iters, ...
169 counts.f, counts.g, counts.H, mem_str, ...
170 S.Rlin_x, S.Rquad_x, char(S.Order_x), S.Rlin_g, S.Rquad_g, char(S.Order_g), conv}; %#ok<AGROW>
171 end
172 end
173 T_TRCG = cell2table(Rows, 'VariableNames', table_vars());
174 disp('=== Trust-Region (CG-Steihaug) ==='); disp(T_TRCG);
175
176 % ----- helper functions -----
177 function v = rand_range(rng2d)
178 v = rng2d(1,1) + (rng2d(1,2) - rng2d(1,1)) * rand;
179 end
180
181 function vars = table_vars()
182 vars = {'Function', 'RunID', 'x0', 'x_final', 'f_final', 'grad_norm', 'Iterations', ...
183 'FuncEvals', 'GradEvals', 'HessEvals', 'MemoryPerIter', ...
184 'Rlin_error', 'Rquad_error', 'Order_error', ...
185 'Rlin_grad', 'Rquad_grad', 'Order_grad', 'Converged'};
186 end
187
188 function S = rate_stats(Xall, xs, gradH)
189 E = vecnorm(Xall - xs, 2, 1);
190 G = arrayfun(@(k) norm(gradH(Xall(:,k))), 1:size(Xall,2));
191 rlin_x = E(2:end) ./ max(E(1:end-1), eps);
192 rquad_x = E(2:end) ./ max(E(1:end-1).^2, eps);
193 rlin_g = G(2:end) ./ max(G(1:end-1), eps);
194 rquad_g = G(2:end) ./ max(G(1:end-1).^2, eps);
195 tail_idx = max(1, ceil(2*numel(rlin_x)/3)) : numel(rlin_x);
196 med = @(v) median(v, 'omitnan');
197 S.Rlin_x = med(rlin_x(tail_idx));
198 S.Rquad_x = med(rquad_x(tail_idx));
199 S.Rlin_g = med(rlin_g(tail_idx));
200 S.Rquad_g = med(rquad_g(tail_idx));
201 S.Order_x = classify_order(S.Rlin_x, S.Rquad_x);
202 S.Order_g = classify_order(S.Rlin_g, S.Rquad_g);
203 end
204
205 function label = classify_order(Rlin, Rquad)
206 if isnan(Rlin)
207 label = "none";
208 return;
209 end
210 if (Rlin < 1e-2) && isfinite(Rquad) && (Rquad > 0) && (Rquad < 1e2)
211 label = "quadratic";
212 return;
213 end

```

```
214 if (Rlin < 1e-1)
215 label = "superlinear";
216 return;
217 end
218 if (Rlin > 0) && (Rlin < 1)
219 label = "linear";
220 else
221 label = "none";
222 end
223 end
224
225 function f = sphere_fun(x)
226 f = x(1).^2 + x(2).^2;
227 end
228
229 function g = sphere_grad(x)
230 g = [2*x(1); 2*x(2)];
231 end
232
233 function H = sphere_hess(~)
234 H = [2 0; 0 2];
235 end
236
237 function f = beale_fun(x)
238 f = (1.5 - x(1) + x(1)*x(2)).^2 + ...
239 (2.25 - x(1) + x(1)*x(2).^2).^2 + ...
240 (2.625 - x(1) + x(1)*x(2).^3).^2;
241 end
242
243 function g = beale_grad(x)
244 A = 1.5 - x(1) + x(1)*x(2);
245 B = 2.25 - x(1) + x(1)*x(2).^2;
246 C = 2.625 - x(1) + x(1)*x(2).^3;
247 dA_dx = -1 + x(2);      dA_dy = x(1);
248 dB_dx = -1 + x(2).^2;    dB_dy = 2*x(1)*x(2);
249 dC_dx = -1 + x(2).^3;    dC_dy = 3*x(1)*x(2).^2;
250 gx = 2*(A*dA_dx + B*dB_dx + C*dC_dx);
251 gy = 2*(A*dA_dy + B*dB_dy + C*dC_dy);
252 g = [gx; gy];
253 end
254
255 function f = gold_fun(x)
256 u = x(1) + x(2) + 1;
257 v = 2*x(1) - 3*x(2);
258 g1 = 19 - 14*x(1) + 3*x(1).^2 - 14*x(2) + 6*x(1)*x(2) + 3*x(2).^2;
259 g2 = 18 - 32*x(1) + 12*x(1).^2 + 48*x(2) - 36*x(1)*x(2) + 27*x(2).^2;
260 A = 1 + u.^2 .* g1;
261 B = 30 + v.^2 .* g2;
262 f = A .* B;
263 end
264
265 function g = gold_grad(x)
266 u = x(1) + x(2) + 1;
267 v = 2*x(1) - 3*x(2);
```

```
268 g1 = 19 - 14*x(1) + 3*x(1).^2 - 14*x(2) + 6*x(1)*x(2) + 3*x(2).^2;
269 g2 = 18 - 32*x(1) + 12*x(1).^2 + 48*x(2) - 36*x(1)*x(2) + 27*x(2).^2;
270 dg1_dx = -14 + 6*x(1) + 6*x(2);
271 dg1_dy = -14 + 6*x(1) + 6*x(2);
272 dg2_dx = -32 + 24*x(1) - 36*x(2);
273 dg2_dy = 48 - 36*x(1) + 54*x(2);
274 A = 1 + u.^2 .* g1;
275 B = 30 + v.^2 .* g2;
276 dA_dx = 2*u*g1 + u.^2*dg1_dx;
277 dA_dy = 2*u*g1 + u.^2*dg1_dy;
278 dB_dx = 4*v*g2 + v.^2*dg2_dx;
279 dB_dy = -6*v*g2 + v.^2*dg2_dy;
280 gx = B .* dA_dx + A .* dB_dx;
281 gy = B .* dA_dy + A .* dB_dy;
282 g = [gx; gy];
283 end
284
285 function f = booth_fun(x)
286 f = (x(1) + 2*x(2) - 7).^2 + (2*x(1) + x(2) - 5).^2;
287 end
288
289 function g = booth_grad(x)
290 g1 = 2*(x(1) + 2*x(2) - 7) + 4*(2*x(1) + x(2) - 5);
291 g2 = 4*(x(1) + 2*x(2) - 7) + 2*(2*x(1) + x(2) - 5);
292 g = [g1; g2];
293 end
294
295 function H = booth_hess(~)
296 H = [10 8; 8 10];
297 end
298
299 function H = num_hess(gradfun, x)
300 h = 1e-6;
301 H = zeros(2,2);
302 e1 = [1; 0];
303 e2 = [0; 1];
304 g1p = gradfun(x + h*e1);
305 g1m = gradfun(x - h*e1);
306 g2p = gradfun(x + h*e2);
307 g2m = gradfun(x - h*e2);
308 H(:,1) = (g1p - g1m) / (2*h);
309 H(:,2) = (g2p - g2m) / (2*h);
310 H = (H + H.') / 2;
311 end
312
313 function [xk, Xall, Dhist, counts] = tr_cg_driver(fun, grad, hess, x0, tol, TR)
314 xk = x0(:);
315 Xall = xk;
316 D = TR.Delta0;
317 Dhist = D;
318 counts = struct('f',0,'g',0,'H',0);
319 for k = 1:TR.maxit
320 gk = grad(xk); counts.g = counts.g + 1;
321 if norm(gk) <= tol, break; end
```

```
322 Bk = hess(xk); counts.H = counts.H + 1;
323 [p, ~] = CG_Steihaug(TR.cg_eps, gk, Bk, D);
324 mk0 = 0; mkp = gk.'*p + 0.5*p.'*Bk*p;
325 denom = -(mkp - mk0); if denom <= 0, denom = eps; end
326 fx = fun(xk); counts.f = counts.f + 1;
327 fnew = fun(xk + p); counts.f = counts.f + 1;
328 rho = (fx - fnew) / denom;
329 accepted = false;
330 if rho >= TR.eta1
331 xk = xk + p; Xall(:, end+1) = xk; %#ok<AGROW>
332 accepted = true;
333 else
334 D = TR.gamma1 * D;
335 end
336 if rho < 0.25
337 D = max(TR.gamma1 * D, 1e-12);
338 elseif (rho > TR.eta2) && (abs(norm(p) - D) <= 1e-12)
339 D = min(TR.gamma2 * D, TR.DeltaMax);
340 end
341 Dhist(end+1) = D; %#ok<AGROW>
342 if accepted
343 gk2 = grad(xk); counts.g = counts.g + 1;
344 if norm(gk2) <= tol, break; end
345 end
346 end
347 end
```